

Discovery Executive Summary

Project: test-context-changes · Generated: 14/08/2026, 16:46:25

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Architecture & Design Analysis	—

1. Architecture & Design Analysis

EXECUTIVE SUMMARY

The Klearcom platform is a voice-quality observability system built as a Laravel 12 monolithic REST API with a React 19 SPA frontend and a parallel Node.js dev-API. While controllers are commendably thin (avg 74 LOC) and the frontend properly uses a shared API client with React Query, the codebase has three systemic architecture issues: (1) zero repository abstractions — all 32 database access points scatter Eloquent calls through controllers and services; (2) domain boundary violations between the Discovery and Connect modules, which share MongoDB collections and cross-read each other's models without interfaces; and (3) critical business logic (the reachability KPI formula) is duplicated in four independent locations across two codebases with no canonical source of truth. The `app/Modules/Connect` and `app/Modules/Discovery` skeleton directories signal an intent to modularize that has not yet begun. These gaps collectively create high change-amplification risk — modifying the reachability calculation, swapping a persistence layer, or extracting a bounded context will each require hunting across multiple files and two languages.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	74 LOC	Good
H2	Missing Service Layer	Controller methods directly accessing models	<10	10–20	>20	13 methods	Moderate
H3	Missing Repository Pattern	Direct DB/ORM access points outside repositories	<10	10–20	>20	32	High Risk
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	Good
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	1 (LegacyDataMapper)	Moderate
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100%	Good
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0	Good
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	6	High Risk
H9	Shared Database Coupling	Data stores shared across domains	<10%	10–30%	>30%	37.5% (3/8)	High Risk
H10	Duplicated Business Logic (additional)	Duplicated business-critical code sites	0–1	2–4	>4	7 sites	High Risk
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	86 LOC	Good
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	2	Good
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0	Good

F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	≤2 levels	Good
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2	Moderate
C1	Anemic Domain Model (context)	Models with domain behaviour (% of total)	>70%	30–70%	<30%	0% (0/4)	High Risk
C2	Empty Module Skeleton (context)	Module directories with actual code	100%	50–100%	<50%	0% (0/2)	High Risk

1.4 Actions Required

Hotspot	Action	Rating	Priority
H3. Missing Repository Pattern	Introduce <code>ConnectMonitorRepository</code> , <code>ConnectCheckResultRepository</code> , <code>DiscoveryJobRepository</code> , <code>DiscoveryNodeRepository</code> interfaces with Eloquent implementations; bind via <code>AppServiceProvider</code> ; inject into services	High Risk	Critical
H10. Duplicated Business Logic	Extract <code>ReachabilityCalculator</code> value object and <code>IvrTreeBuilder</code> service as single sources of truth; update all 7 call sites across PHP and JS codebases	High Risk	Critical
H8. Domain Boundary Violations	Split <code>RealTimeTestService</code> into <code>DiscoveryTestService</code> and <code>ConnectTestService</code> ; create published query interfaces for cross-domain reads; refactor <code>DashboardController</code> and <code>LegacyReportController</code> to use them	High Risk	High
H9. Shared Database Coupling	Split shared MongoDB collections into domain-owned repositories with typed document schemas; or add anti-corruption layer between domains and shared collections	High Risk	High
C1. Anemic Domain Model	Add domain behaviour to models: <code>ConnectMonitor::computeReachability()</code> , <code>DiscoveryJob::transitionTo()</code> , <code>DiscoveryNode::buildTree()</code> ; enforce state-machine transitions	High Risk	High
H2. Missing Service Layer	Create <code>ConnectService</code> , <code>DiscoveryService</code> , <code>DashboardService</code> ; move all business logic out of 13 controller methods into services	Moderate	Medium
C2. Empty Module Skeleton	Either populate <code>app/Modules/Connect/</code> and <code>app/Modules/Discovery/</code> with actual domain code, or remove the empty directories to avoid misleading structure	High Risk	Medium
H5. Shared Utility Abuse	Replace <code>extract()</code> in <code>LegacyDataMapper</code> with explicit array access; register as a service via DI; consider replacing with a dedicated DTO or Transformer	Moderate	Low
F5. Legacy / Inconsistent Patterns	Convert <code>LegacyMonitorPoller</code> class component to function component with hooks; convert <code>LegacyDashboardWidget</code> to React Query; add Error Boundary at layout level	Moderate	Low

1.5 Expected Outcomes

- **Testable business logic:** Extracting `ReachabilityCalculator` , `IvrTreeBuilder` , and domain services means every business rule can be unit-tested without HTTP requests or a live database.
- **Single source of truth for KPIs:** The duplicated reachability formula converges to one canonical implementation, eliminating the risk of inconsistent alert states across API endpoints.
- **Independent domain evolution:** With repository interfaces and bounded-context boundaries enforced via `app/Modules/` , the Discovery and Connect domains can evolve, scale, and eventually extract independently.
- **Safe persistence swaps:** Repository abstractions let the team add read replicas, caching layers, or migrate from MariaDB to Aurora without touching business logic.
- **Onboarding clarity:** Removing the empty `Modules/` skeleton (or populating it) and standardizing frontend patterns (all function components, all React Query) means new developers see a consistent architecture rather than two competing paradigms.